claude-windows / dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3\subagents\workflows\wf_b0e273c7-38a\agent-a824d7f7e00953fb3.jsonl`
- SHA-256: `29ae056cb7e3ceb3ffef71e83c79b13bdaafbaebb21693e9364e51457d6cdf73`
- Source modified: `2026-06-21T08:05:12+00:00`
- Imported at: `2026-07-05T16:48:22+00:00`
- project: `wf_b0e273c7-38a`
- session_id: `dc4e53de-0f32-4ae9-8e0e-d6c7d74779f3`

Transcript

1. user (2026-06-21T08:02:26.841Z)

```
You are reviewing milestone M2 of COMPUTE_DECOUPLED_SERVING in the repo at
C:/Users/avidu/Projects/badminton-highlight-indexer.
Spec: docs/COMPUTE_DECOUPLED_SERVING/README.md §7 row M2 = "input_backend/input_root; wire main
CLI/API input through StorageRef.parse_uri (local=identity); storage-fake tests". M2 must be
SAFE and
default-OFF: with the default config (storage.input_backend='local', input_root=''), the loaded
input
path must be byte-identical to before ? a bare/local path is read in place with NO copy. A
bucket/Drive
URI (gs://, s3://, gdrive://) is fetched to a local scratch file before processing.

The change (read all of these + git diff against origin/master):
- backend/config/models.py: StorageConfig gains input_backend + input_root.
- backend/storage/ref.py: resolve_input_uri() (precedence: explicit scheme > anchored local path
>
  input_root join > input_backend prefix > bare-local).
- backend/storage/__init__.py: acquire_local_input() (local=identity, remote=mkdtemp+fetch) +
  input_is_local().
- backend/main.py: _execute_processing() (API/job path) + the CLI --video-path block both
  acquire-then-use the local path (keeping the original URI for the DB record + report);
  /api/process skips the local os.path.exists() gate for a non-local URI.
- tests: tests/test_input_source.py (new) + 3 tests in tests/test_api_endpoints.py.

Reference the worker's already-shipped pattern (backend/worker/__main__.py:91-105 cmd_infer) and
the
path-safety helpers (backend/utils/paths.py: validate_input_path / allowed_video_root). Ground
EVERY
claim in code with file:line. Give the exact triggering input + wrong-vs-right behaviour. Prefer
FEWER,
HIGH-CONFIDENCE findings. A clean bill is a valid, good result.

Four reviewers produced these raw findings (JSON):
[
  {
    "title": "Unknown-scheme video_path (https://, ftp://, file://) crashes /api/process with
HTTP 500 instead of a clean 4xx",
    "severity": "major",
    "confidence": "high",
    "location": "backend/main.py:278 (storage.input_is_local call, outside the try/except at
lines 270-273); root cause backend/storage/ref.py:68 (parse_uri raises ValueError on unknown
scheme)",
    "detail": "M2 added `if storage.input_is_local(req.video_path, ...)` at backend/main.py:278,
OUTSIDE the try/except ValueError that wraps validate_input_path (lines 270-273). input_is_local
```

-> resolve_input_uri -> parse_uri, and parse_uri raises `ValueError(\"unsupported storage URI scheme: ...\")` for any scheme not in (local,s3,gcs,gdrive,gs). In the DEFAULT, non-hosted deployment (security.allowed_video_root defaults to None, confirmed via AppConfig()), validate_input_path passes (no null byte, no root), then the unhandled ValueError from input_is_local propagates -> FastAPI 500. There is no global exception_handler registered (grep confirmed), so the 500 is what the client gets. Reproduced end-to-end with TestClient(raise_server_exceptions=False): POST /api/process {video_path:'https://user:pass@host/x.mp4'} -> 500; same for 'ftp://h/x.mp4' and 'file:///etc/passwd'. Before M2, /api/process never called parse_uri on the raw path, so these inputs hit `os.path.exists(...)` -> a clean 404. M2 turns that clean 404 into a 500 (information-leaking stack trace, and a DoS-ish crash vector for any reachable client). The 3 new tests cover local-identity, local-404, and gs://-fetch but NOT the unknown-scheme branch, so this slipped through.",
    "fix": "Move the input_is_local existence check inside a try/except, or extend the existing try block (lines 270-280) so a ValueError from input_is_local also maps to HTTPException(400). Mirror the same guard in the CLI block (backend/main.py:~592, `storage.input_is_local(args.video_path, ...)`). Simplest: wrap both validate_input_path AND the input_is_local/exists gate in one `try: ... except ValueError as e: raise HTTPException(400, str(e))`. Add a test asserting POST /api/process with 'https://...'/'ftp://...' returns 400 (not 500).",
    "lens": "security"
  },
  {
    "title": "Explicit remote URI (gs://, s3://, gdrive://) triggers a network fetch with no per-deployment opt-in gate",
    "severity": "minor",
    "confidence": "high",
    "location": "backend/storage/__init__.py:54-58 (acquire_local_input) + backend/storage/ref.py:87-88 (resolve_input_uri precedence rule 1: explicit scheme wins)",
    "detail": "resolve_input_uri rule 1 returns any explicit-scheme URI as-is regardless of config, so acquire_local_input fetches it whenever ref.backend != 'local'. The config flags input_backend/input_root only govern how BARE names resolve; they do NOT gate whether an explicit gs://attacker-bucket/x.mp4 is honored. Verified: acquire_local_input('gs://attacker-bucket/x.mp4', None) (default config that did NOT opt into remote input) still calls storage.fetch on that URI. In the default single-user local deployment, /api/process accepts a gs://...//s3://...//gdrive://... video_path (input_is_local=False -> existence gate skipped, job queued) and the worker later performs the egress. Threat surface is bounded by who can reach the API (local single-user today), but the spec frames M2's default as 'read in place, no copy' for local ? there is no config that says 'this deployment only accepts local inputs', so a deployment that never intended remote I/O can still be steered into an outbound fetch (unexpected egress / mild SSRF) by a single request field. This is a design-gap, not a crash.",
    "fix": "Gate remote acquisition on opt-in: e.g. when input_backend=='local' AND input_root=='' (the default), have acquire_local_input/input_is_local reject (or refuse to fetch) an explicit non-local scheme rather than silently fetching it ? require the operator to set input_backend/input_root (or an explicit allow_remote_input flag) before any bucket/Drive URI is honored. At minimum document that any reachable client of /api/process can initiate egress regardless of input config.",
    "lens": "security"
  },
  {
    "title": "gdrive:// (and local://) keys with ../ traversal escape the mount/cwd; now reachable from API/CLI video_path via M2 input wiring",
    "severity": "minor",
    "confidence": "medium",
    "location": "backend/storage/gdrive.py:92-97 (_local_ref: rel=ref.key.lstrip('/'); os.path.join keeps '..') + backend/storage/local.py:23-27 (_path joins under base_dir with no containment); reached via backend/storage/__init__.py:57 acquire_local_input",
    "detail": "For gdrive://../../etc/passwd, parse_uri yields key '../../etc/passwd'; GDriveStorage._local_ref lstrips only leading '/' and os.path.join preserves the '..' segments, then LocalStorage._path joins under the mount base_dir with no realpath/containment check -> the read escapes the Drive 'My Drive' root. Similarly acquire_local_input('local://../../etc/passwd') returns '../../etc/passwd' (the literal local:// prefix stripped), which the segmenter opens relative to cwd. This is pre-existing backend behavior, but M2 newly routes API/CLI video_path through these backends, so it is now reachable from the request boundary. In HOSTED mode (allowed_video_root set) validate_input_path rejects

these (it realpaths the RAW string and fails containment ? verified), so the practical exposure is non-hosted deployments with a gdrive mount, or any caller of acquire_local_input that skips validate_input_path. Note also a latent mismatch: validate_input_path checks the RAW 'local://...'/'gdrive://...' string while acquire_local_input opens the SCHEME-STRIPPED remainder; I could not construct a working raw-passes-but-stripped-escapes case (the extra 'local:'/'gdrive:' path segment makes both escape together), so I am not claiming a containment bypass ? only flagging the two-strings-validated-vs-opened divergence as fragile.",
    "fix": "Add containment/normalization in the path-keyed backends: in LocalStorage._path / GDriveStorage._local_ref, reject keys whose normalized form escapes base_dir (os.path.normpath + a base_dir-prefix assert, like resolve_under_root). Independently, prefer validating the RESOLVED local path (the exact string acquire_local_input returns) rather than the raw URI, so the validated string == the opened string.",
    "lens": "security"
  },
  {
    "title": "Default-config local input is byte-identical through both API and CLI ? M2 guarantee holds",
    "severity": "nit",
    "confidence": "high",
    "location": "backend/storage/ref.py:86-98, backend/storage/__init__.py:49-65, backend/main.py:159-160,214,597-637",
    "detail": "Clean bill on the core M2 safety contract. Verified empirically that under the default config (input_backend='local', input_root='') resolve_input_uri() and acquire_local_input() return the SAME string object for every path form in the lens ? \"C:\\\\v\\\\x.mp4\", \"x.mp4\", \"C:/v/x.mp4\", \"/data/x.mp4\", \"rel/sub/x.mp4\" ? with no normalization and no copy/temp dir (acquire returns ref.key which is the unaltered input string; resolve branch 5 / branch 2 returns s unchanged). (a) byte-identity proven. (b) Wiring is complete and correct: every file-reading consumer uses local_video ? compute_video_id (main.py:160 / CLI:599), registry.run_all_with_context (166 / 604), segmenter.process_video (214 / 637) ? while every provenance use keeps the original URI ? db.add_video (179/190 / CLI:609), create_job (289), emit_indexer_report video_path (251 / CLI:652). No file-reading use still leaks the URI; no provenance use was over-replaced. The async path is sound: process_video stores req.video_path (the URI) in the job row, and backend/api/job_worker.py:64-69 _job_to_request reconstructs ProcessRequest(video_path=job['video_path']) and calls _execute_processing with NO extra os.path.exists gate, so a gs:// job flows to the acquire-then-fetch in the worker (confirmed by test_process_nonlocal_uri_skips_exists_and_fetches). (c) compute_video_id (utils/hashing.py:23-33) opens and MD5s the file at local_video ? fetched bytes for remote (correct content-identity, matching worker/__main__.py:104-105), the same in-place file for local. (d) The --trim (main.py:556) and --debug-clip (main.py:566) utilities still pass args.video_path directly; this is an acceptable scope boundary (diagnostic/dev utilities outside 'main CLI/API input', and byte-identical under the default config) ? not a correctness gap for M2. Tests: tests/test_input_source.py (25) + tests/test_api_endpoints.py (17) + 126 storage/worker tests all pass; 1096 tests collect clean.",
    "fix": "No change required for the M2 default-OFF / byte-identical guarantee. Optional: add an inline note that --trim/--debug-clip are an intentional scope boundary (they don't honor input_backend/input_root).",
    "lens": "backcompat"
  },
  {
    "title": "acquire_local_input docstring claims cache-hygiene tooling reclaims the scratch copy, but nothing does",
    "severity": "minor",
    "confidence": "high",
    "location": "backend/storage/__init__.py:56-57,64-65",
    "detail": "For a REMOTE input, acquire_local_input does tempfile.mkdtemp(prefix=\"rally-input-\", dir=scratch_parent) with scratch_parent defaulting to None, so the scratch dir lands under the SYSTEM temp (%TEMP% / /tmp) and is never deleted by the function. The docstring asserts: \"The scratch copy is left in place (a pure intermediate, regenerable from the source) ? same as the worker's cmd_infer pull, so cache-hygiene tooling reclaims it.\" That reclaim claim is not substantiated: tools/cleanup_caches.py:40-52 only walks the project's cache locations and classifies dirs whose name startswith(\"tmp\") as a purgeable temp dir (line 50) ? it does NOT scan the system temp dir and the \"rally-input-\" prefix does not match \"tmp\", so these dirs accumulate until the OS clears /tmp (and on Windows %TEMP% is not auto-cleared). This is a latent disk leak on the non-default (remote-input) path only and an attribution-discipline issue (the docstring describes a reclaim path that isn't wired). It does NOT affect the default

config (local = identity, no scratch created) and the worker already ships the identical pattern (prefix=\"rally-worker-\"), so it is pre-existing, not introduced here.",
    "fix": "Either (a) soften the docstring to state the scratch dir is left under the system temp and relies on OS temp reclamation (drop the 'cache-hygiene tooling reclaims it' claim), or (b) actually wire reclamation ? e.g. place the scratch under a project-local dir the cleanup tool scans and/or have cleanup_caches.py match the rally-input-/rally-worker- prefixes, or register an atexit/finally cleanup once the file-reading consumers are done with local_video.",
    "lens": "backcompat"
  },
  {
    "title": "local:// input is wrongly rejected as missing ? existence gate stats the raw URI, not the resolved local key",
    "severity": "minor",
    "confidence": "high",
    "location": "backend/main.py:278-280 (API) and backend/main.py:594 (CLI)",
    "detail": "For a `local://<path>` input, `storage.input_is_local(req.video_path)` returns True (parse_uri treats `local://` as the local backend; the M2 test tests/test_input_source.py:59 even asserts `input_is_local(\"local://x.mp4\") is True`), so the existence gate runs. But the gate stats the RAW string `req.video_path` (`os.path.exists(\"local://C:/.../x.mp4\")`) instead of the resolved key that `acquire_local_input` would return (the `local://` prefix stripped). Verified empirically: a real on-disk file passed as `local://<abs>` gives `input_is_local=True`, `os.path.exists(raw)=False`, but `os.path.exists(acquire_local_input(raw))=True`. Triggering input: `POST /api/process {\"video_path\":\"local:///data/x.mp4\"}` (or the CLI `--video-path local:///data/x.mp4`) where `/data/x.mp4` exists. Wrong behaviour: HTTP 404 \"Video file not found\" (API) / \"[FAIL] Video file not found\" then `return` (CLI). Right behaviour: the file should be accepted and processed, since `local://` is a documented local input form (ref.py:6, parse_uri). Note this is a pre-existing weakness for the raw `os.path.exists` (M2 did not regress the bare-path default path, which is byte-identical), but M2 newly introduces `input_is_local` returning True for `local://`, making the gate/resolution mismatch a live inconsistency. Bare and absolute local paths ? today's actual inputs ? are unaffected; only the explicit `local://` scheme is hit.",
    "fix": "Resolve once and stat the resolved local key. E.g. compute `local_video = storage.acquire_local_input(req.video_path, sc)` (it is identity/no-copy for any local input) and gate on `os.path.exists(local_video)` when `input_is_local` is True; reuse that same `local_video` for hashing/validation. Apply symmetrically at main.py:594/597 (CLI) so `--video-path local://...` behaves identically. Add a `local://<tmp file>` 404-then-202 test alongside the existing gs:// case in tests/test_api_endpoints.py.",
    "lens": "correctness"
  },
  {
    "title": "Worker path (cmd_infer) bypasses resolve_input_uri, so input_root/input_backend config is silently honored on the main/API path but ignored on the worker path",
    "severity": "nit",
    "confidence": "high",
    "location": "backend/worker/__main__.py:101 vs backend/storage/__init__.py:49-65 (acquire_local_input)",
    "detail": "The two fetch entry points resolve inputs differently. The API/CLI path (`backend.main`) routes through `acquire_local_input` -> `resolve_input_uri`, which applies `storage.input_backend`/`input_root` (a bare name `x.mp4` with `input_root=gs://bucket/v` becomes `gs://bucket/v/x.mp4`). The decoupled-compute worker `cmd_infer` calls `storage.fetch(args.video_uri, ...)` directly on the raw `video_uri` with NO `resolve_input_uri`, so the same bare name + the same `input_root` config stays a LOCAL identity passthrough on the worker. This is not a bytes-divergence for fully-qualified URIs (the dispatcher always hands the worker a complete `gs://...` URI, which both paths treat identically), and it is arguably correct that the worker takes pre-resolved URIs. But it means `input_root`/`input_backend` are config knobs that work on one entry point and are inert on the other, which can surprise an operator who sets them globally. Low impact for M2 (the worker is the M6+ Cloud Run dispatch target, not the API job drainer ? the API drainer is `_execute_processing`, which IS instrumented). Flagging for the design record, not as a blocker.",
    "fix": "Either (a) document on `StorageConfig.input_root`/`input_backend` that they apply to the main/API+CLI entry points only and that the worker expects fully-resolved URIs, or (b) when the worker later needs config-relative inputs, route `cmd_infer` through `acquire_local_input` too so both entry points share one resolution. No change needed for M2 to ship.",

```
    "lens": "correctness"
  },
  {
    "title": "New unit-test file tests/test_input_source.py is UNTRACKED ? risks shipping M2
with no unit coverage",
    "severity": "blocker",
    "confidence": "high",
    "location": "tests/test_input_source.py (whole file); git status: `??
tests/test_input_source.py`",
    "detail": "`git status --short` shows tests/test_input_source.py as untracked (`??`), not
staged and NOT present in `git diff origin/master` (the diff stat lists only the 4 src files +
tests/test_api_endpoints.py). This file is the spec deliverable for M2 ('tests:
tests/test_input_source.py (new)') and contains the entire unit suite for the three new public
helpers: resolve_input_uri (all 5 precedence branches, parametrized 16 cases), input_is_local
(local/bucket/input_root/Pydantic-config), and acquire_local_input (identity/relative-
identity/remote-fetch/input_root-join/never-fetch-for-local). With the project's explicit-path
staging convention (CLAUDE.md: 'Stage explicit paths (git add <files>) ? never git add -A'), it
is easy to commit the 5 modified files and forget this brand-new untracked file. If that
happens, M2 ships with the helper layer entirely unverified ? only the 3 API-path tests in
test_api_endpoints.py would remain, none of which cover resolve_input_uri's branch table or
acquire_local_input's identity-vs-fetch logic directly. Triggering input: cut the PR staging
only the diffed files. Wrong behaviour: ~30 assertions silently absent; right behaviour: the
file is tracked and runs in CI.",
    "fix": "Explicitly `git add tests/test_input_source.py` before committing (verify with `git
ls-files tests/test_input_source.py` returning the path, and confirm it appears in `git diff
--cached --stat`). Re-run `python run_all_tests.py` after staging to confirm the file is
collected by CI.",
    "lens": "tests"
  },
  {
    "title": "CLI --video-path block (the second M2 call-site) has no test ? only the API path
is covered",
    "severity": "major",
    "confidence": "high",
    "location": "backend/main.py:588-637 (CLI block: input_is_local gate :594,
acquire_local_input :597, original-URI persisted :609, local_video to segmenter :637)",
    "detail": "M2 modifies TWO callers symmetrically: _execute_processing (API/job,
main.py:159-216) and the CLI --video-path block (main.py:588-637). The 3 new tests
(test_api_endpoints.py:351-394) and the API integration assertions exercise ONLY the
/api/process path via TestClient ? confirmed by grepping all tests/: no test invokes
backend.main.main() with --video-path (the only main()-with-argv tests are scripts/ablation.py
and golden fixtures, never backend.main). The CLI block carries its own independent copy of the
M2 logic: the `storage.input_is_local(args.video_path, storage_cfg) and not os.path.exists(...)`
short-circuit (:594), the acquire-then-hash (:597-599), keeping args.video_path in db.add_video
while processing local_video (:609 vs :637). None of this is pinned. Regressions that would pass
CI: (a) dropping the `input_is_local` guard at :594 so a `gs://` CLI input wrongly prints
'[FAIL] Video file not found' and returns; (b) accidentally passing args.video_path (the URI)
instead of local_video to validators/segmenter at :604/:637; (c) the short-circuit ordering
breaking so a missing LOCAL file no longer fast-fails. Triggering input: `python -m backend.main
--video-path gs://bucket/x.mp4` (must fetch+process) and `--video-path /no/such/file.mp4` (must
print FAIL + return).",
    "fix": "Add a CLI test that drives backend.main.main() with monkeypatched sys.argv (pattern
already in tests/test_ablation.py:501): one case with a non-local URI (monkeypatch storage.fetch
+ a capturing segmenter; assert the segmenter receives a LOCAL path, NOT the gs:// URI, and that
db.get_video stores the original URI), and one with a missing local path asserting early return
without calling the segmenter. This mirrors the 3 API tests onto the second, currently-untested
call-site.",
    "lens": "tests"
  },
  {
    "title": "Hosted-mode (allowed_video_root set) rejection of a non-local URI is asserted in
no test",
    "severity": "major",
    "confidence": "high",
    "location": "backend/main.py:269-280 + comment :277; behaviour in
```

```
backend/utils/paths.py:24-34 (resolve_under_root)",
    "detail": "The code comment at main.py:277 makes an explicit safety claim: 'a configured
allowed_video_root (hosted mode) rejects a non-local URI.' I verified it holds ?
validate_input_path('gs://b/x.mp4', allowed_root='/srv/videos') raises, as do s3://, gdrive://,
local:// ? but ONLY incidentally: realpath('gs://b/x.mp4') resolves the URI as a cwd-relative
path ('.../gs:/b/x.mp4'), which fails the commonpath containment check. This is load-bearing
security behaviour (in hosted mode a tenant must not be able to point the engine at an arbitrary
bucket the operator didn't sanction) yet it rests on a path-normalization side effect, not an
explicit non-local check, and NO test exercises it. The env fixture
(test_api_endpoints.py:27-34) uses a bare AppConfig() with allowed_video_root unset, so every M2
test runs in non-hosted mode. A future refactor of resolve_under_root or validate_input_path
could silently stop rejecting bucket URIs in hosted mode with zero test failures. Triggering
input: global_config.security.allowed_video_root='/srv/videos'; POST /api/process
{video_path:'gs://bucket/x.mp4'}.",
    "fix": "Add a test that sets allowed_video_root on the config and asserts POST /api/process
with a gs:// (and gdrive://) video_path returns 400 (rejected), while a local path under the
root returns 202. Pin the security contract rather than relying on incidental realpath
behaviour.",
    "lens": "tests"
  },
  {
    "title": "Non-local-URI test does not assert the original URI is preserved in the DB/job
record",
    "severity": "minor",
    "confidence": "high",
    "location": "tests/test_api_endpoints.py:383-394
(test_process_nonlocal_uri_skips_exists_and_fetches) vs backend/main.py:179,190,289",
    "detail": "The spec and the main.py:157 comment require keeping the source URI for the DB
record + report while processing the fetched local copy. The non-local test correctly asserts
the segmenter receives a fetched LOCAL path (not the gs:// URI), but never asserts the
contrapositive: that the persisted record keeps the ORIGINAL gs:// URI. main.py persists
req.video_path (the URI) at create_job (:289) and add_video (:179/:190), using local_video only
for the file-reading consumers. If a regression swapped local_video into add_video/create_job,
the DB would record an ephemeral scratch path (e.g. /tmp/rally-input-xxxx/x.mp4) that vanishes
after the job ? losing provenance for the data-flywheel/report ? and every current test would
still pass. This is the 'keep the original URI for the DB record + report' half of the M2
contract going unverified.",
    "fix": "In test_process_nonlocal_uri_skips_exists_and_fetches, after the job completes,
assert db.get_video(video_id)['video_path'] (or the job row's video_path) ==
'gs://bucket/videos/x.mp4', confirming the source URI ? not the scratch path ? is persisted.",
    "lens": "tests"
  }
]

Adjudicate as a skeptical senior engineer. Independently re-check each against the actual code;
DROP false positives and scope-creep nits; MERGE duplicates; keep only genuinely actionable
items for a SAFE, default-OFF M2. Return the final must-fix / should-fix list ordered by
severity with concrete fixes. If sound and ship-ready, say so with empty findings.
```

2. assistant (2026-06-21T08:02:29.105Z)

I'll re-check each finding against the actual code. Let me start by reading the core files and the git diff.

3. user (2026-06-21T08:02:30.464Z)

```
 backend/config/models.py    | 10 ++++++++
 backend/main.py             | 33 +++++++++++++++++++++-------
 backend/storage/__init__.py | 45 ++++++++++++++++++++++++++++++++++++--
 backend/storage/ref.py      | 30 ++++++++++++++++++++++++
 tests/test_api_endpoints.py | 58 ++++++++++++++++++++++++++++++++++++++++++++++++++++
 5 files changed, 165 insertions(+), 11 deletions(-)
---STATUS---
 M backend/config/models.py
 M backend/main.py
```

```
 M backend/storage/__init__.py
 M backend/storage/ref.py
 M tests/test_api_endpoints.py
?? artifacts/
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
?? tests/test_input_source.py
```

4. assistant (2026-06-21T08:02:32.438Z)

Let me read the key files.

5. user (2026-06-21T08:02:33.391Z)

```
1       """Storage value objects + URI parsing for the pluggable storage layer.
2
3       A ``StorageRef`` is a backend-agnostic pointer to one object. URIs:
4
5           <bare path>                 -> local   (today's behaviour; e.g. C:/v/x.mp4,
/data/x.mp4)
6           local://<path>              -> local   (local:///abs or local://rel/p)
7           s3://<bucket>/<key>         -> s3       (AWS + R2 + DO Spaces + MinIO via
endpoint_url)
8           gcs://<bucket>/<key>        -> gcs      (gs:// accepted, normalized to gcs://)
9           gdrive://<path>             -> gdrive   (path under your locally-MOUNTED Google
Drive "My Drive")
10
11      No cloud SDKs are imported here ? this module is pure stdlib so ``import
backend.storage``
12      stays cheap.
13      """
14      from __future__ import annotations
15
16      import re
17      from dataclasses import dataclass
18      from datetime import datetime
19      from typing import Optional
20
21      _SCHEME_RE = re.compile(r"^([A-Za-z][A-Za-z0-9+.\-]*)://")
22      _KNOWN = ("local", "s3", "gcs", "gdrive")
23
24
25      @dataclass(frozen=True)
26      class StorageStat:
27          """Lightweight object metadata (uniform across backends)."""
28          size: int
29          modified: Optional[datetime] = None
30          etag: Optional[str] = None
31          content_type: Optional[str] = None
32
33
34      @dataclass(frozen=True)
35      class StorageRef:
36          """A backend + a location. ``bucket`` is the S3/GCS bucket; for ``local`` and
``gdrive`` it is
37          empty and ``key`` carries the path (a filesystem path, or a path under the mounted
Drive)."""
38          backend: str
39          bucket: str
40          key: str
41          uri: str
42
43          @property
44          def name(self) -> str:
45              return self.key.rstrip("/").rsplit("/", 1)[-1]
46
```

```
47
48      def parse_uri(uri: str) -> StorageRef:
49          """Parse a storage URI (or a bare filesystem path) into a ``StorageRef``.
50
51          A string with no ``scheme://`` prefix (including Windows ``C:\\...`` paths, which
52          have no ``//``) is treated as a local filesystem path ? preserving today's
behaviour.
53          """
54          s = str(uri)
55          m = _SCHEME_RE.match(s)
56          if not m:
57              return StorageRef("local", "", s, f"local://{s}")
58          scheme = m.group(1).lower()
59          rest = s[m.end():]
60          if scheme == "gs":
61              scheme = "gcs"
62          if scheme in ("local", "gdrive"):
63              # Path-keyed, no bucket: local FS, or a path under the mounted Google Drive "My
Drive".
64              return StorageRef(scheme, "", rest, s)
65          if scheme in _KNOWN:
66              bucket, _, key = rest.partition("/")
67              return StorageRef(scheme, bucket, key, s)
68          raise ValueError(f"unsupported storage URI scheme: {scheme}:// (in {s!r})")
69
70
71      def resolve_input_uri(raw: str, *, input_backend: str = "local", input_root: str = "")
-> str:
72          """Resolve a user-supplied input reference into a storage URI
(COMPUTE_DECOUPLED_SERVING M2).
73
74          Precedence (first match wins):
75            1. An explicit ``scheme://`` on ``raw`` (``gs://``, ``s3://``, ``local://``,
``gdrive://``) ? as-is.
76            2. An ABSOLUTE local path (``/data/x.mp4``, ``C:\\?``, ``C:/?``) ? as-is (always
local).
77            3. A bare/relative name + a non-empty ``input_root`` ? joined under it
78               (``"gs://bucket/videos"`` + ``"x.mp4"`` ? ``"gs://bucket/videos/x.mp4"``).
79            4. A bare/relative name + ``input_backend != "local"`` ? prefixed (``"gcs"`` +
``"bucket/x"`` ?
80               ``"gcs://bucket/x"``).
81            5. Otherwise ? ``raw`` unchanged (a relative/bare LOCAL path ? today's behaviour).
82
83          So the default (``input_backend="local"``, ``input_root=""``) is an identity
passthrough for every
84          path used today, preserving byte-for-byte behaviour.
85          """
86          s = str(raw)
87          if _SCHEME_RE.match(s):           # 1. explicit scheme wins
88              return s
89          # 2. an anchored LOCAL path is always local, never joined. Detect this OS-
INDEPENDENTLY ?
90          #    os.path.isabs is platform- AND Python-version-dependent (on Windows + py3.13 a
POSIX-style
91          #    "/data/x" is NOT absolute). Anchored == a leading "/" or "\\", or a Windows
drive prefix "X:".
92          if s.startswith(("/", "\\")) or (len(s) >= 2 and s[1] == ":" and s[0].isalpha()):
93              return s
94          if input_root:                    # 3. join a relative name under the configured
root
95              return input_root.rstrip("/") + "/" + s.lstrip("/")
96          if input_backend and input_backend != "local":   # 4. prefix a bucket/key with the
declared backend
97              return f"{input_backend}://{s}"
98          return s                          # 5. bare/relative local path ? unchanged
```

99

6. user (2026-06-21T08:02:33.599Z)

```
1       """Pluggable storage layer: local (default) + S3/GCS first-class buckets + Google Drive.
2
3       URI-driven (see ``ref.parse_uri``). The package import is cheap ? cloud SDKs are
imported
4       lazily, only when a non-local bucket URI is actually used (install the ``storage-s3`` /
5       ``storage-gcs`` extras). The default ``local`` backend needs no third-party deps and
preserves
6       today's behaviour byte-for-byte; ``gdrive://`` likewise needs none ? it reads a locally
MOUNTED
7       Google Drive (Google Drive for Desktop) as plain files.
8
9       Worker/dispatcher entry points speak URIs via the thin helpers ``fetch`` / ``put`` /
10      ``list_uris`` / ``get_storage``; ``local://`` (and bare paths) make ``fetch`` an
identity
11      passthrough. See ``docs/DECOUPLED_COMPUTE/``.
12      """
13      from __future__ import annotations
14
15      import os
16      import tempfile
17      from typing import Any, Dict, Iterator, Optional
18
19      from backend.storage.base import StorageBackend
20      from backend.storage.ref import StorageRef, StorageStat, parse_uri, resolve_input_uri
21
22      __all__ = [
23          "StorageBackend", "StorageRef", "StorageStat", "parse_uri", "resolve_input_uri",
24          "get_storage", "fetch", "put", "list_uris",
25          "acquire_local_input", "input_is_local",
26      ]
27
28
29      def _storage_dict(storage_config: Any) -> Dict[str, Any]:
30          """Coerce a StorageConfig (Pydantic) / dict / None into a plain settings dict."""
31          if storage_config is None:
32              return {}
33          if hasattr(storage_config, "model_dump"):
34              return storage_config.model_dump()  # type: ignore[no-any-return]
35          return dict(storage_config)
36
37
38      def input_is_local(raw: str, storage_config: Any = None) -> bool:
39          """True if ``raw`` resolves (under the input config) to a LOCAL filesystem path ?
i.e. a
40          plain on-disk file the caller can stat/validate directly. False for a bucket / Drive
URI
41          that must be fetched first. Lets callers keep their local-only checks
(os.path.exists,
42          allowed_video_root) for the local case and skip them for a remote source."""
43          sc = _storage_dict(storage_config)
44          uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
45                                  input_root=sc.get("input_root", ""))
46          return parse_uri(uri).backend == "local"
47
48
49      def acquire_local_input(raw: str, storage_config: Any = None, *,
50                              scratch_parent: "str | None" = None) -> str:
51          """Resolve ``raw`` (a path or storage URI) to a LOCAL file path ready for
processing.
52
53          Local / bare paths are returned UNCHANGED (identity passthrough, no copy ? today's
```

```
behaviour
54          byte-for-byte). A bucket / Drive source (``gs://``/``s3://``/``gdrive://``, or a
bare name under a
55          non-local ``input_root``/``input_backend``) is downloaded into a fresh scratch dir
and the local
56          path returned. The scratch copy is left in place (a pure intermediate, regenerable
from the
57          source) ? same as the worker's ``cmd_infer`` pull, so cache-hygiene tooling reclaims
it."""
58          sc = _storage_dict(storage_config)
59          uri = resolve_input_uri(raw, input_backend=sc.get("input_backend", "local"),
60                                  input_root=sc.get("input_root", ""))
61          ref = parse_uri(uri)
62          if ref.backend == "local":
63              return ref.key  # identity ? no temp, no copy, byte-identical to passing the
path directly
64          scratch = tempfile.mkdtemp(prefix="rally-input-", dir=scratch_parent)
65          return fetch(uri, os.path.join(scratch, ref.name or "in.mp4"), config=sc)
66
67
68      def _backend_class(name: str):
69          """Lazy-resolve a backend class so cloud SDKs aren't imported until needed."""
70          if name == "local":
71              from backend.storage.local import LocalStorage
72              return LocalStorage
73          if name == "s3":
74              from backend.storage.s3 import S3Storage
75              return S3Storage
76          if name == "gcs":
77              from backend.storage.gcs import GCSStorage
78              return GCSStorage
79          if name == "gdrive":
80              from backend.storage.gdrive import GDriveStorage
81              return GDriveStorage
82          raise ValueError(f"unknown storage backend: {name!r} (expected
local|s3|gcs|gdrive)")
83
84
85      def get_storage(backend: Optional[str] = None, *,
86                      config: Optional[Dict[str, Any]] = None, **overrides) -> StorageBackend:
87          """Construct a backend. ``backend`` (or ``config['backend']``, default 'local')
selects
88          the implementation; its per-backend settings come from ``config[backend]`` merged
with
89          ``overrides``. Settings carry NO secrets ? only endpoints/regions/paths/source-
selectors."""
90          cfg = config or {}
91          name = backend or cfg.get("backend", "local")
92          settings: Dict[str, Any] = dict(cfg.get(name, {}) or {})
93          settings.update(overrides)
94          return _backend_class(name)(**settings)
95
96
97      def fetch(uri: str, dst: "str | os.PathLike[str] | None" = None, *,
98                config: Optional[Dict[str, Any]] = None, overwrite: bool = False) -> str:
99          """Resolve ``uri`` to a local file path. For ``local`` this is an identity
passthrough
100         (no copy); for s3/gcs it downloads to ``dst``."""
101         ref = parse_uri(uri)
102         return get_storage(ref.backend, config=config).fetch_to_local(ref, dst,
overwrite=overwrite)
103
104
105     def put(local_path: "str | os.PathLike[str]", uri: str, *,
106             config: Optional[Dict[str, Any]] = None, content_type: Optional[str] = None) ->
```

```
str:
107         """Upload/copy ``local_path`` to ``uri``; return ``uri``."""
108         ref = parse_uri(uri)
109         get_storage(ref.backend, config=config).put(local_path, ref,
content_type=content_type)
110         return uri
111
112
113     def list_uris(prefix_uri: str, *, config: Optional[Dict[str, Any]] = None) ->
Iterator[str]:
114         """Yield the URI of every object under ``prefix_uri``."""
115         ref = parse_uri(prefix_uri)
116         for r in get_storage(ref.backend, config=config).list(ref):
117             yield r.uri
118
```

7. assistant (2026-06-21T08:02:35.534Z)

Now the main.py around the API and CLI call sites.

8. user (2026-06-21T08:02:36.566Z)

```
150         (after this function has already restored the pre-run segments, see below).
151         """
152         notify = progress or (lambda stage: None)
153
154         notify("hashing")
155         # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
(identity ?
156         # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
original
157         # req.video_path (the source URI) is kept for the DB record + report; only the file-
reading
158         # consumers (hash / validators / segmenter) use the resolved local path.
159         local_video = storage.acquire_local_input(req.video_path, getattr(global_config,
"storage", None))
160         video_id = compute_video_id(local_video)
161         was_reingest = db_instance.get_video(video_id) is not None
162
163         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
164         notify("validating")
165         logger.info(f"Running validations for {req.video_path}...")
166         val_results, val_context = registry.run_all_with_context(local_video, global_config)
167         failures = [r for r in val_results if not r.passed]
168
169         # typed AppConfig: attribute access for the default segmenter (with safe fallback)
170         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
171         seg_name = req.segmenter or default_seg
172         segmenter_actual = None
173         segmentation_ran = False
174         response: Optional[Dict[str, Any]] = None
175         try:
176             if failures:
177                 # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
178                 # status; it no longer destroys the video's prior good segments.
179                 db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
180                 response = {
181                     "video_id": video_id,
182                     "status": "failed",
183                     "message": "Video failed validation checks.",
184                     "results": [r.model_dump() for r in val_results],
185                 }
```

```
186                 return response
187
188             # 2. Run segmenter & indexer
189             logger.info("[API] Video passed checks. Segmenting and indexing...")
190             db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
191
192             segmenter_cls = segmenter_registry.get(seg_name)
193             if not segmenter_cls:
194                 # Submit-time validation already 400s unknown names; this is the defensive
195                 # in-worker path (e.g. config default pointing at an unregistered
segmenter).
196                 available = list(segmenter_registry.list_available().keys())
197                 response = {
198                     "video_id": video_id,
199                     "status": "failed",
200                     "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
201                     "results": [r.model_dump() for r in val_results],
202                     "play_segments": [],
203                 }
204                 return response
205
206             logger.info(f"[API] Using segmenter: {seg_name}")
207             notify(f"segmenting ({seg_name})")
208             segmenter_actual = seg_name
209             # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
210             # if the run yields segments; otherwise drop the new partial rows and keep the
old.
211             play_wm, cand_wm = db_instance.segment_watermarks(video_id)
212             segmenter = segmenter_cls(db_instance, global_config)
213             try:
214                 result = segmenter.process_video(video_id, local_video, req.player_pool,
req.max_frames)
215             except Exception:
216                 # A raising segmenter must not leave half-written rows: restore the pre-run
217                 # state (same destroy-after-confirm contract as the Failure branch), then
let
218                 # the job worker record the failure.
219                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
220                 raise
221             segmentation_ran = True
222
223             if isinstance(result, Failure):
224                 logger.error(f"[API] Segmenter failed: {result.message}")
225                 db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
226                 response = {
227                     "video_id": video_id,
228                     "status": "failed",
229                     "message": f"Segmenter '{seg_name}' failed: {result.message}",
230                     "results": [r.model_dump() for r in val_results],
231                     "play_segments": [],
232                 }
233                 return response
234
235             segments = result.value if hasattr(result, "value") else result
236             notify("finalizing")
237             _finalize_reingest(video_id, segments, play_wm, cand_wm)
238
239             response = {
240                 "video_id": video_id,
241                 "status": "success",
242                 "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
```

```
                "results": [r.model_dump() for r in val_results],
                "play_segments": segments,
            }
            return response
        finally:
            # Always emit the per-video observability report (next to the video, or to
            # report.output_dir). Never raises. Surface the paths in the response.
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths


    @app.post("/api/process", status_code=202)
    def process_video(req: ProcessRequest):
        """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.

        Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
        runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
        worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
        """
        allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
        try:
            validate_input_path(req.video_path, allowed_root=allowed_root)
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        # M2: the local-existence fast-fail only applies to a LOCAL input. A bucket/Drive
URI can't be
        # cheaply os.path.exists()'d ? its existence is verified when the worker fetches it
(a fetch miss
        # fails the job loudly). validate_input_path above still runs for every input: its
null-byte guard
        # always applies, and a configured allowed_video_root (hosted mode) rejects a non-
local URI.
        if storage.input_is_local(req.video_path, getattr(global_config, "storage", None)):
            if not os.path.exists(req.video_path):
                raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
        if req.segmenter and not segmenter_registry.get(req.segmenter):
            available = list(segmenter_registry.list_available().keys())
            raise HTTPException(
                status_code=400,
                detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
            )

        job_id = uuid.uuid4().hex
        if not db_instance.create_job(job_id, req.video_path, req.segmenter,
                                      req.player_pool, req.max_frames):
            raise HTTPException(status_code=500, detail="Failed to persist job record.")
        _get_executor().submit(_drain_due_jobs)
        return JSONResponse(
            status_code=202,
            content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
        )


    @app.get("/api/jobs/{job_id}")
```

```
300     def get_job(job_id: str):
301         """Job state: {status, progress, result, reports}. `status` = execution state
302         (queued|running|done|failed); the pipeline verdict is result["status"]."""
303         job = db_instance.get_job(job_id)
304         if not job:
305             raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
306         result = job.get("result")
307         return {
308             "job_id": job["id"],
309             "status": job["status"],
```

9. assistant (2026-06-21T08:02:38.423Z)

Now the CLI block.

10. user (2026-06-21T08:02:39.652Z)

```
540                 out_name = os.path.basename(out_path)
541             else:
542                 out_dir = args.output_dir
543                 out_path = os.path.join(out_dir, out_filename)
544                 out_name = out_filename
545 
546             os.makedirs(out_dir, exist_ok=True)
547             print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
548 
549             # global_config isn't initialized this early; load the typed config so the trim
550             # clip carries the same configured stitching paddings + watermark as a real
551             # highlight (watermark default-on).
552             stitching = load_app_config().stitching
553             compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
554                                      end_padding=stitching.end_padding,
watermark=stitching.watermark)
555             try:
556                 res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
557                 print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
558             except Exception as e:
559                 print(f"[ERROR] Failed to compile trimmed segment: {e}")
560             return
561 
562         if args.debug_clip is not None:
563             if not args.video_path:
564                 print("[ERROR] Please provide --video-path to debug a specific clip.")
565                 return
566             run_cli_diagnose_clip(args.video_path, args.debug_clip)
567             return
568 
569         # Initialize Global Config and DB
570         global global_config, db_instance
571         global_config = load_app_config()  # returns typed AppConfig
572 
573         # The CLI --output-dir overrides the configured output directory. It now lives
574         # on the typed model (OutputConfig.output_dir), so every consumer ? including
575         # /api/compile ? reads the same value instead of a write-only runtime hack.
576         global_config.output.output_dir = args.output_dir
577         os.makedirs(global_config.output.output_dir, exist_ok=True)
578 
579         db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
580         db_instance = Database(db_path)
581 
582         # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
583         # the executor is in-process. Mark them failed so pollers see a terminal state.
```

```
        swept = db_instance.fail_stale_jobs()
        if swept:
            logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")

        # CLI processing mode (no web UI needed)
        if args.video_path and not args.server:
            print(f"[CLI] Processing video file: {args.video_path}")
            # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
            # The local-existence fast-fail only applies to a local input; a remote fetch
fails loudly.
            storage_cfg = getattr(global_config, "storage", None)
            if storage.input_is_local(args.video_path, storage_cfg) and not
os.path.exists(args.video_path):
                print(f"[FAIL] Video file not found: {args.video_path}")
                return
            local_video = storage.acquire_local_input(args.video_path, storage_cfg)

            video_id = compute_video_id(local_video)
            was_reingest = db_instance.get_video(video_id) is not None

            # Validate (with-context variant surfaces ffprobe_meta for the report)
            print("[CLI] Validating...")
            val_results, val_context = registry.run_all_with_context(local_video,
global_config)
            failures = [r for r in val_results if not r.passed]

            # Save video status
            status = "failed" if failures else "processed"
            db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])

            seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
            segmenter_actual = None
            segmentation_ran = False
            try:
                print("\n" + "="*40)
                print("VALIDATION SUMMARY")
                print("="*40)
                for r in val_results:
                    status_symbol = "[PASS]" if r.passed else "[FAIL]"
                    print(f"{status_symbol} {r.validator_name}: {r.message}")
                print("="*40 + "\n")

                if failures:
                    print("[FAIL] Video failed checks. Indexing halted.")
                    return

                # Index
                segmenter_cls = segmenter_registry.get(seg_name)
                if not segmenter_cls:
                    print(f"[FAIL] Segmenter '{seg_name}' not found.")
                    return

                print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
                segmenter_actual = seg_name
                play_wm, cand_wm = db_instance.segment_watermarks(video_id)
                segmenter = segmenter_cls(db_instance, global_config)
                result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
                segmentation_ran = True

                if isinstance(result, Failure):
```

```
641                     print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
642                     print("[FAIL] Indexing halted for video.")
643                     segments = []
644                 else:
645                     segments = result.value if hasattr(result, "value") else result
646                     print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
647                 # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
648                 _finalize_reingest(video_id, segments, play_wm, cand_wm)
649             finally:
650                 # Always emit the per-video observability report. Never raises.
651                 paths = emit_indexer_report(
652                     db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
653                     val_results=val_results, val_context=val_context,
654                     segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
655                     was_reingest=was_reingest, segmentation_ran=segmentation_ran,
656                 )
657                 if paths:
658                     print(f"[REPORT] Technical report : {paths.get('technical_md')}")
659                     print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
```

11. assistant (2026-06-21T08:02:42.486Z)

Now let me check `validate_input_path` and `paths.py`, plus the new tests.

12. user (2026-06-21T08:02:43.002Z)

```
1       """Path-safety helpers for the API boundary (untrusted request fields)."""
2       import os
3       from typing import Optional
4
5
6       def safe_output_filename(name: str) -> str:
7           """Validate an output filename is a BARE name (no dir part, no traversal, not
absolute).
8
9           `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is
absolute,
10          and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-
write
11          primitive. Returns the name unchanged when safe; raises ValueError otherwise.
12          """
13          if not name or not name.strip():
14              raise ValueError("output_filename must not be empty")
15          if "\x00" in name:
16              raise ValueError("output_filename must not contain a null byte")
17          if "/" in name or "\\" in name:
18              raise ValueError(f"output_filename must not contain a path separator: {name!r}")
19          if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
20              raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
21          return name
22
23
24      def resolve_under_root(path: str, root: str) -> str:
25          """Return the real path, asserting it is contained under `root` (raises ValueError
if not)."""
26          rp = os.path.realpath(path)
27          rr = os.path.realpath(root)
28          try:
29              contained = os.path.commonpath([rp, rr]) == rr
30          except ValueError:  # different drives on Windows
31              contained = False
32          if not contained:
```

```
33              raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
34          return rp
35
36
37      def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
38          """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.
39
40          With no allowed_root (default, single-user local mode) only the null-byte check
applies,
41          so the local 'point it at any of my videos' workflow is preserved. Setting an
42          allowed_root (a future hosted/tenant deployment) enforces containment.
43          """
44          if "\x00" in path:
45              raise ValueError("video_path must not contain a null byte")
46          if allowed_root:
47              resolve_under_root(path, allowed_root)
48
```

13. assistant (2026-06-21T08:02:45.393Z)

Now let me verify the key claims empirically. First, the unknown-scheme crash (finding 1) and the new tests.

14. user (2026-06-21T08:02:46.100Z)

```
340     class _CaptureSeg:
341         """Records the video_path the segmenter actually received (M2: must be a LOCAL
path)."""
342         last_path = None
343
344         def __init__(self, db, cfg):
345             self.db = db
346
347         def process_video(self, video_id, video_path, *a, **k):
348             _CaptureSeg.last_path = video_path
349             sid = self.db.add_play_segment(video_id, 0.0, 5.0)
350             return [{"id": sid, "start_time": 0.0, "end_time": 5.0, "duration": 5.0,
"metadata": {}}]
351
352
353     def test_process_local_input_passes_identity_path(env):
354         """Default (local) input must reach the segmenter as the EXACT same path ? identity,
no copy
355         (the M2 zero-behaviour-change guarantee for today's local workflow)."""
356         client, db, tmp_path, mp = env
357         vid = _video(tmp_path)
358         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
359         mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
360         _CaptureSeg.last_path = None
361         r = client.post("/api/process", json={"video_path": str(vid), "segmenter": "x"})
362         assert r.status_code == 202
363         assert _CaptureSeg.last_path == str(vid)
364
365
366     def test_process_local_missing_file_still_404(env):
367         """A LOCAL input that doesn't exist still fast-fails 404 ? the existence gate is
preserved
368         for local paths (only non-local URIs skip it)."""
369         client = env[0]
370         r = client.post("/api/process", json={"video_path": "/no/such/local/file.mp4"})
371         assert r.status_code == 404
372
373
374     def test_process_nonlocal_uri_skips_exists_and_fetches(env):
375         """A gs:// input is NOT 404'd at submit (can't os.path.exists a bucket object); the
```

```
worker
376         fetches it to a local scratch file and the segmenter sees that LOCAL path, not the
URI."""
377         client, db, tmp_path, mp = env
378
379         def fake_fetch(uri, dst=None, *, config=None, overwrite=False):
380             with open(dst, "wb") as fh:
381                 fh.write(b"fetched-video-bytes")
382             return dst
383
384         mp.setattr(m.storage, "fetch", fake_fetch)
385         mp.setattr(m.registry, "run_all_with_context", lambda *a, **k: (_pass(), {}))
386         mp.setattr(m.segmenter_registry, "get", lambda name: _CaptureSeg)
387         _CaptureSeg.last_path = None
388         r = client.post("/api/process", json={"video_path": "gs://bucket/videos/x.mp4",
"segmenter": "x"})
389         assert r.status_code == 202                              # NOT 404 ? non-local skips
the exists gate
390         job = client.get(f"/api/jobs/{r.json()['job_id']}").json()
391         assert job["status"] == "done"
392         assert _CaptureSeg.last_path is not None
393         assert not _CaptureSeg.last_path.startswith("gs://")     # a fetched LOCAL path, not
the URI
394         assert os.path.basename(_CaptureSeg.last_path) == "x.mp4"
395
```

15. assistant (2026-06-21T08:03:01.764Z)

Now let me empirically verify finding 1 (unknown scheme ? 500) and finding 6 (local:// rejected).

16. user (2026-06-21T08:03:05.755Z)

```
https://h/x.mp4 -> RAISES ValueError unsupported storage URI scheme: https:// (in
'https://h/x.mp4')
ftp://h/x.mp4 -> RAISES ValueError unsupported storage URI scheme: ftp:// (in 'ftp://h/x.mp4')
file:///etc/passwd -> RAISES ValueError unsupported storage URI scheme: file:// (in
'file:///etc/passwd')
---local:// finding 6---
raw= local://C:/Users/avidu/AppData/Local/Temp/tmpm7hww4e7.mp4
input_is_local= True
os.path.exists(raw)= False
acquire= C:/Users/avidu/AppData/Local/Temp/tmpm7hww4e7.mp4
exists(acquire)= True
```

17. assistant (2026-06-21T08:03:08.768Z)

Both confirmed. Finding 1: unknown scheme raises ValueError. The call at main.py:278 is OUTSIDE the try/except (270-273). Let me confirm via TestClient that it produces a 500.

18. user (2026-06-21T08:03:18.060Z)

```
13:33:17 [INFO] httpx: HTTP Request: POST http://testserver/api/process "HTTP/1.1 500 Internal
Server Error"
13:33:17 [INFO] httpx: HTTP Request: POST http://testserver/api/process "HTTP/1.1 500 Internal
Server Error"
https://h/x.mp4 -> 500
ftp://h/x.mp4 -> 500
file:///etc/passwd -> 500
```

19. assistant (2026-06-21T08:03:23.747Z)

Finding 1 confirmed: unknown scheme ? HTTP 500. Let me check what pre-M2 behavior was ? did `os.path.exists` give a clean 404 before? Before M2, the code did `if not

os.path.exists(req.video_path)` unconditionally. `os.path.exists('https://h/x.mp4')` is False ? 404. So M2 regressed a clean 404 into a 500. This is a real regression and a blocker/major.

Now let me verify the CLI path has the same issue (it calls `input_is_local` at :594, also not wrapped). And check finding 8/9/10 (test coverage). Let me check whether the CLI block has any test.

20. user (2026-06-21T08:03:25.767Z)

```
tests\test_api_endpoints.py:18

Found 18 total occurrences across 1 file.
```

21. user (2026-06-21T08:03:26.872Z)

```
Found 1 file
tests\test_worker.py
```

22. user (2026-06-21T08:03:29.673Z)

```
119:    rc = wmain.main([
187:    rc = wmain.main(["infer", "--video-uri", str(video), "--out-uri", str(tmp_path / "o"),
220:    rc = wmain.main(["train", "--corpus-uri", str(tmp_path / "corpus"), "--out-uri",
str(out_dir),
275:    rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
299:    rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(tmp_path /
"out"),
```

23. assistant (2026-06-21T08:03:35.862Z)

Confirmed: those are worker tests (`wmain.main`), not `backend.main.main` with `--video-path`. So finding 8 (CLI block untested) is correct ? no test drives the CLI processing block.

Now let me verify finding 10 (non-local test doesn't assert URI preserved in DB). Looking at the test (lines 374-394), it asserts the segmenter got a local path but never checks `db.get_video(...)['video_path'] == 'gs://...'`. Confirmed by reading.

Now let me assess findings 2, 3, 5, 7 quickly and the untracked-test blocker (finding 8 in JSON list is the untracked file).

Let me verify the untracked test file is real (git status already showed `?? tests/test_input_source.py`). Confirmed. This is a real risk given the explicit-path staging convention.

Let me check finding 5 (docstring cleanup claim) against cleanup_caches.py briefly.

24. user (2026-06-21T08:03:36.380Z)

```
38
39
40      def classify(name: str, is_dir: bool, location: str) -> Tuple[str, bool, bool]:
41          """Classify a cache entry by name. Returns (category, regenerable,
default_purge)."""
42          low = name.lower()
43          if is_dir:
44              if low.endswith("_frames"):
45                  return ("frame cache", True, True)
46              if low.endswith("_wasbcache"):
47                  return ("detector resume cache", True, False)  # opt-in via --include-
wasbcache
48              if low.startswith("chunks_") and low.endswith("_handoff"):
49                  return ("ai handoff clips", True, True)
50              if low.startswith("tmp"):
51                  return ("temp dir", True, True)
```

```
52              return ("other dir", False, False)
53          # files
```

25. assistant (2026-06-21T08:03:44.969Z)

Confirmed finding 5: `cleanup_caches.py` only matches dir names starting with "tmp" (line 50), and only walks project cache locations ? not system temp. The `rally-input-` prefix won't match, and the dirs land in system temp by default. The docstring's "cache-hygiene tooling reclaims it" claim is inaccurate. This is a minor doc-accuracy issue (attribution discipline matters per project norms), default-OFF path only.

Now let me evaluate findings 2 and 3 (remote egress gate / traversal) for scope. These are pre-existing backend behaviors that M2 newly routes through. Finding 2 (no opt-in gate for explicit remote URI) ? the spec explicitly says a bucket/Drive URI is fetched. That's intended M2 behavior, and threat is bounded to single-user local today. It's a design-gap note, not a must-fix for SAFE default-OFF (default config still byte-identical). I'll fold a brief mention but not as must-fix.

Finding 3 (traversal in gdrive://local:// keys) ? pre-existing, and hosted mode rejects it via validate_input_path; medium confidence, reporter themselves couldn't construct a working bypass. Scope-creep for M2. Drop as must-fix.

Let me run the test suite to confirm green and that the new file collects.

26. user (2026-06-21T08:03:59.496Z)

```
.....................................                                    [100%]
42 passed in 11.06s
```

27. assistant (2026-06-21T08:04:03.099Z)

Now let me verify finding 9 (hosted-mode rejection of non-local URI is untested but the comment claims it). Let me confirm the behavior holds and that no test sets allowed_video_root.

28. user (2026-06-21T08:04:05.498Z)

```
gs://b/x.mp4 -> rejected
s3://b/x.mp4 -> rejected
gdrive://x.mp4 -> rejected
local://x.mp4 -> rejected
```

29. assistant (2026-06-21T08:04:18.202Z)

Confirmed: hosted-mode rejection holds (incidentally, via realpath), and no test sets `allowed_video_root`. This is load-bearing security behavior resting on a normalization side-effect with zero test pinning ? a legitimate should-fix test gap.

Now I have enough to adjudicate. Let me finalize.

Adjudication summary:
- **Finding 1 (unknown scheme ? 500)**: CONFIRMED real regression. Major. Must-fix.
- **Finding 6 (local:// wrongly 404'd)**: CONFIRMED. The gate stats raw URI. Minor correctness, newly live via M2's `input_is_local`. Should-fix. (Same root fix family as #1 ? both are the existence-gate not using the resolved path.)
- **Finding 8 (untracked test file)**: CONFIRMED `??`. Blocker given staging convention. Must-fix (stage it).
- **Finding "CLI block untested" (major)**: CONFIRMED no test drives `backend.main.main` --video-path. Should-fix.
- **Finding "hosted-mode untested" (major)**: CONFIRMED. Should-fix test.
- **Finding 10 (URI-preservation not asserted)**: CONFIRMED. Minor test gap.
- **Finding 5 (docstring reclaim claim)**: CONFIRMED inaccurate. Minor.
- **Finding 4 (clean-bill)**: nit, drop from findings (positive).
- **Finding 2 (remote egress no opt-in)**: intended M2 behavior, bounded; design-note not must-fix. Fold into #1's context lightly. Drop as standalone.

- **Finding 3 (traversal)**: pre-existing, hosted-mode rejects, no working bypass. Drop (scope-creep).
- **Finding 7 (worker bypasses resolve)**: nit/design-record, correct but not actionable for M2. Drop.

I'll merge #1 and #6 partly (both existence-gate using raw instead of resolved path), but they have distinct symptoms so I'll keep them separate but note the shared fix.

30. user (2026-06-21T08:04:58.622Z)

Structured output provided successfully

31. assistant (2026-06-21T08:05:12.337Z)

Adjudication complete. Summary of what I kept, merged, and dropped for M2 (SAFE, default-OFF):

Core M2 safety contract holds. The default config (input_backend='local', input_root='') is a verified identity passthrough ? resolve_input_uri/acquire_local_input return the input unchanged with no copy for every local path form, and the 42 M2 tests pass. The clean-bill finding was confirmed; I dropped it from the findings list as a positive rather than an issue.

Must-fix / should-fix kept (7):
1. **Unknown-scheme ? HTTP 500 (major)** ? real regression, verified via TestClient that `https://`/`ftp://`/`file://` now 500 where they previously 404'd. Affects both the API (:278) and CLI (:594) call-sites.
2. **Untracked test file (major)** ? `?? tests/test_input_source.py` confirmed; under the explicit-path staging convention it can be left out of the commit, shipping the helper layer unverified.
3. **local:// wrongly 404'd (minor)** ? same root cause as #1 (gate stats raw URI not resolved key); verified.
4. **CLI block untested (minor)** ? confirmed no test drives `backend.main.main` --video-path.
5. **Hosted-mode rejection untested (minor)** ? load-bearing security behavior resting on an incidental realpath side-effect, zero coverage.
6. **URI-preservation not asserted (minor)** ? provenance half of the contract unverified.
7. **Docstring reclaim claim inaccurate (nit)** ? confirmed `cleanup_caches.py` won't reclaim the `rally-input-` scratch dirs.

Dropped as false-positive / scope-creep:
- Remote-egress opt-in gate (finding 2): the spec explicitly defines fetching a bucket/Drive URI as intended M2 behavior; default config is still byte-identical and threat is bounded to single-user local. Design-note, not a SAFE-M2 must-fix.
- gdrive://local:// traversal (finding 3): pre-existing backend behavior, hosted-mode validate_input_path rejects it, and the reporter could not construct a working bypass.
- Worker bypasses resolve_input_uri (finding 7): correct observation but a design-record nit; the worker takes pre-resolved URIs and is not the M2 API/CLI entry point.

Relevant files: `C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py`, `backend/storage/ref.py`, `backend/storage/__init__.py`, `backend/utils/paths.py`, `tests/test_api_endpoints.py`, `tests/test_input_source.py`, `tools/cleanup_caches.py`.